

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 5 : Session – 4 : SLO – 1 | BFS Vs DFS

■ I. Complete the Pseudocode — DFS (using stack)

1. Initialize an empty STACK
2. Mark all vertices as UNVISITED
3. Push the starting vertex onto the stack
4. While the stack is not empty:
 - a. Pop a vertex from the stack
 - b. If the vertex is UNVISITED:
 - i. Mark it as visited
 - ii. Push all its ADJACENT vertices onto the stack

■ II. Complete the Pseudocode — BFS (using queue)

1. Initialize an empty QUEUE
2. Mark all vertices as UNVISITED
3. Enqueue the starting vertex
4. While the queue is not empty:
 - a. Dequeue a vertex from the queue
 - b. If the vertex is UNVISITED:
 - i. Mark it as visited
 - ii. Enqueue all its ADJACENT vertices

■ III. Arrange the Steps in Correct Order

- DFS: b -> c -> f -> a -> e -> d
- (mark start unvisited; push start; while stack not empty: pop; mark visited; push adjacent unvisited)
- BFS: b -> a -> f -> c -> d -> e
- (mark start unvisited; enqueue start; while queue not empty: dequeue; mark visited; enqueue adjacent)

■ IV. Spot the Error

- Error: the snippet uses a QUEUE with push/dequeue/enqueue — that is BFS logic, not DFS.
- Correct it to: use a STACK — 1. Push starting vertex onto STACK; 2. While stack is not empty:
 - a. POP a vertex; b. If unvisited: mark as visited; PUSH all adjacent vertices.

■ V. Reasoning Activity

- Use BFS. BFS explores the graph level by level from Person A, so the first time Person B is reached,
- the path found has the smallest number of hops. DFS may wander deep along one branch and would not
- guarantee the shortest connection.